



AULA 2

Bancos de dados: projeto conceitual e lógico, SQL DDL (esquemas) e SQL LMD (instâncias)



Objetivos de aprendizagem

Ao final desta aula, você irá:

- Entender o processo de modelagem conceitual de dados.
- Compreender mapeamentos para esquemas relacionais.
- Conhecer a fração DDL da linguagem SQL.
- Praticar SQL DML para popular bases com dados.
- Investigar controle de restrições de integridade.
- Obter noções básicas sobre o conceito de transações.



Que história é essa?

Esta segunda aula tem como foco a construção de bancos de dados relacionais, desde o projeto conceitual. O aluno terá também a oportunidade de aprender o mapeamento entre modelos conceituais e lógicos, de forma a obter esquemas de bancos relacionais apropriados. Vamos abordar brevemente a teoria de normalização, que permite não somente obter, mas também validar a qualidade de esquemas. Introduziremos, também, a noção de transações, um conceito fundamental para sistemas de bancos de dados na prática.

Continuaremos nesta aula com a apresentação da linguagem SQL, desta vez enfatizando os comandos de criação e manutenção das relações em um esquema, além de aprender a instanciar o banco com dados e verificar as restrições de integridade.

Revisão

Na aula anterior trabalhamos com alguns esquemas de bancos de dados relacionais já existentes, a base de dados sobre Carros (tabelas: Automóveis, Revendedoras, Consumidores e Negócios) e a base de dados de Cervejas (tabelas: Gostam e Vendem). Foram escolhas apropriadas para podermos introduzir os conceitos básicos de modelos de dados e o modelo relacional, as restrições de integridade estruturais mais relevantes, que envolvem chaves primárias e estrangeiras, e podermos realizar consultas com a linguagem SQL, exemplificando os esquemas de Carros e Cervejas. Entretanto, como foram projetados e implementados esses bancos de dados que utilizamos?

De fato, é importante entender bem o processo de construção para que seja possível criarmos nossos próprios sistemas e bancos de dados para aplicações de interesse prático.

Desde que o modelo de dados relacional foi proposto, nos anos de 1970, houve questionamentos sobre quais e quantas tabelas deveriam ser criadas para cada sistema computacional. Uma única tabela já atenderia os requisitos? Se optarmos por múltiplas tabelas, quantas e quais devem ser propostas? Vamos discutir algumas estratégias que permitem obter um conjunto considerado adequado de relações para sistemas relacionais em aplicações reais.

Temos, a seguir, uma visão geral do processo envolvido no projeto de um banco de dados relacional. Clique nas caixas para abrir as informações:



Interativo

Sobre um fundo cinza claro, esquema com cinco retângulos, dois à esquerda e três à direita. Todos eles têm um sinal de mais na base central, que começa na parte interna e termina na externa. À esquerda, os retângulos 1 e 2 estão um abaixo do outro e têm o fundo preto. À direita, os retângulos também estão um abaixo do outro. O 3 com o fundo bordô, o 4 com o fundo azul e o 5 com o fundo marrom. No retângulo 1: a frase, em branco: Análise e levantamento de requisitos de dados e funcionais. No 2: a frase, em branco: Dependências funcionais e formas normais (modelo relacional). No 3, em branco: Projeto conceitual. No 4, em branco: Projeto lógico. No 5, em preto: Projeto físico. No esquema, 4 flechas. A primeira sai do Retângulo 1 em direção ao Retângulo 3. A segunda sai do Retângulo 2 em direção ao Retângulo 4. A terceira sai do Retângulo 3 em direção ao Retângulo 4. E a quarta, do Retângulo 4 ao Retângulo 5.

Processo de modelagem conceitual



Seguindo o processo ilustrado anteriormente, antes de projetarmos um banco de dados, para qualquer modelo lógico, não apenas relacional, sugere-se elaborar um modelo estritamente conceitual que contemple todos os dados do banco e a maneira como que esses dados se relacionam.

O modelo de entidades e relacionamentos (MER), proposto em 1976 por Peter Chen, é um dos mais utilizados pelos profissionais de mercado e, certamente, o processo de modelagem conceitual mais difundido. Na década de 1990, com a popularização do paradigma de orientação a objetos (OO) para linguagens de programação, houve um esforço para a consolidação de métodos e modelos existentes para projeto de sistemas que resultou na *Unified Modeling Language* (UML). Especificamente, é o diagrama de classes da UML que poderia servir como arcabouço para definir adequadamente esquemas conceituais.

Fique ligado

Tanto o MER como a UML têm recursos com muitas opções para caracterização dos conjuntos de dados e seus relacionamentos, algo necessário para a implementação de um banco de dados que dará suporte a um sistema de informações específico. De certa forma, a UML é uma linguagem bem mais complexa. Não somente é útil para modelagem de dados, mas também serve como base para desenvolvimento de programas completos. Como aqui estamos interessados apenas na modelagem conceitual com foco nos dados, **vamos considerar apenas o MER ao longo do texto.**

Vale lembrar que, como vimos na primeira aula, temos dois níveis de representação abstrata no processo de modelagem. Nesse caso, temos o **modelo** de entidades de relacionamentos, genérico e válido para projetos conceituais diversos, e podemos ter múltiplos **esquemas** de entidades e relacionamentos, que são particulares para cada problema ou aplicação em si.

Entidades e atributos

A modelagem por entidades e relacionamentos tem alto grau de abstração e foi proposta com o intuito de capturar a semântica de aplicações do mundo real. É um modelo simples, intuitivo, mas formal, que define como representar dados e relacionamentos entre estes, com regras e especificações que devem ser satisfeitas para que a modelagem possa ser considerada correta.

Basicamente, temos três tipos de objetos a modelar:

- as entidades;
- os relacionamentos; e
- seus atributos.

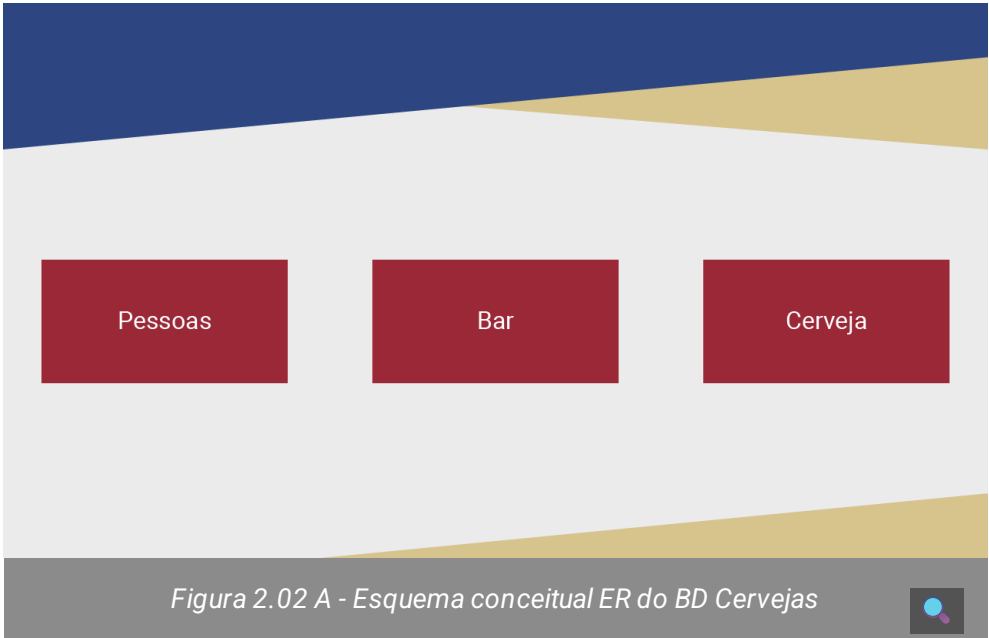
Como forma de linguagem, adota-se na prática uma representação visual conhecida como **diagrama de entidades e relacionamentos**

(DER), que facilita o uso da modelagem e as trocas entre usuários-desenvolvedores e usuários não técnicos.

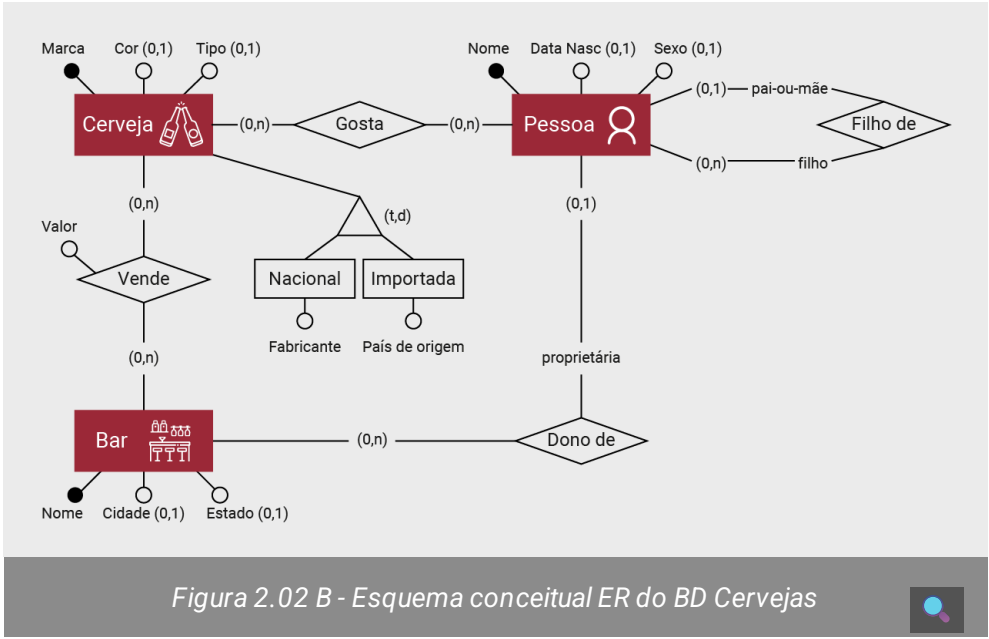


Naturalmente, temos como o primeiro conceito a noção de **entidade**. Como qualquer outra abstração, definir entidade não é trivial. Podemos dizer que se trata de algo (ou uma “coisa”) que pode ser inequivocamente identificado por um conjunto de propriedades ou características em comum.

Em um DER, as entidades são representadas por meio de quadrados ou retângulos. Seja o DER da Figura 2.02B, que diz respeito a um esquema conceitual ER sobre cervejas, desenhado com o apoio da ferramenta brModelo. Observe que se trata de uma versão ampliada e mais detalhada que o banco de dados das cervejas contemplado na primeira aula. Temos três entidades principais:



Buscamos modelar uma aplicação em que registramos quais pessoas gostam de determinadas cervejas e quais bares vendem cervejas, de determinadas marcas (nacionais ou importadas), por preços específicos. Mais adiante comentaremos sobre as entidades Nacional e Importada, que estão diretamente associadas à entidade Cerveja.



As propriedades relevantes de cada instância em uma entidade são representadas por meio de **atributos**. É fundamental entender que não existem entidades sem atributos, pois são estes que os definem, e não o nome da entidade, que serve apenas para facilitar a identificação em um diagrama. Por exemplo, como as pessoas que apreciam cervejas têm as mesmas características (atributos) que as



peessoas que são proprietárias de bares, não faria sentido termos duas entidades diferentes.

Os atributos de uma entidade são representados no DER por seus nomes, dispostos na parte externa do retângulo que representa uma entidade, ao lado de pequenos círculos. Em algumas representações visuais, a lista de atributos é incluída dentro do retângulo da entidade, mas, aqui, vamos adotar a notação mais convencional, que também é utilizada pela ferramenta [brModelo](#).

Na Figura 2.02B, vemos que a **entidade Bar tem três atributos: Nome, Cidade e Estado**, que permitem especificar cada um dos bares desse banco de dados particular. Por simplificação, consideramos que não há dois bares com mesmo nome para esse sistema em construção. O atributo Nome aparece com o círculo preenchido de cor preta, significando que se trata de um **atributo identificador**, que permite identificar todas as instâncias, ou ocorrências, dessa entidade. Cada ocorrência representa um bar diferente.

Atributos identificadores têm interpretação similar à das super-chaves (SK) em tabelas do modelo relacional, ou seja, em um MER, não há exigência de minimalidade, como no caso de chaves primárias (PK), do modelo relacional.

Os demais atributos (Cidade e Estado) da entidade Bar complementam as informações sobre cada ocorrência na entidade Bar. Veja que os atributos são apresentados no DER acompanhados por dois valores entre parênteses: o primeiro informa se o atributo é opcional (valor 0) ou obrigatório (valor 1); o segundo diz respeito ao fato de ser mono (valor 1) ou multivalorado (valor N > 1). Os atributos identificadores são todos implicitamente definidos como obrigatórios e monovalorados. No caso, no exemplo da Figura 2.02B, temos que tanto Cidade como Estado, na entidade Bar, são opcionais, ou seja, podem assumir algum valor, mas também podem ser desconhecidos.

Fique ligado

Neste DER em particular, todos os atributos, de todas as entidades, inclusive os identificadores, são monovalorados. Os atributos ditos multivalorados são aqueles que podem representar mais de um valor, de mesma natureza, em seu conteúdo. Poderia ser o caso de ocorrências na entidade Cerveja cujo atributo Tipo, por exemplo, tivesse mais de um valor.

Existe outra categoria de atributo, chamada de estruturada ou composta, que também contém mais de um valor, como a multivalorada, porém de natureza (semântica) distinta. Não temos também exemplos de atributos compostos na Figura 2.02B, mas, apenas como ilustração, é como se, no caso da entidade Pessoa, a data de nascimento não fosse representada por um único valor, e sim um atributo subdividido em três outros atributos separados: dia, mês e ano.

Relacionamentos e hierarquias entre entidades

Como o próprio nome diz, em um MER temos entidades e **relacionamentos**. Os losangos utilizados no DER representam os relacionamentos entre ocorrências de entidades ou, genericamente, relacionamentos entre entidades. Na Figura 2.02B, estão

representados diversos relacionamentos, de tipos distintos, que serão detalhados a seguir.



Se observarmos o diagrama sobre cervejas, temos um relacionamento denominado “Dono de” entre as entidades Pessoa e Bar. Ele significa que determinada pessoa pode ser dona, ou proprietária, de algum bar. Relacionamentos são descritos adicionalmente por meio de **cardinalidades mínimas e máximas**. Nesse relacionamento “Dono de” entre Pessoas e Bares, temos a cardinalidade (0,1) próxima de Pessoa: o primeiro valor (cardinalidade mínima) é referente à obrigatoriedade (valor 1), ou não (valor 0), de se ter algum proprietário de determinado bar. Nesse caso, a modelagem explicita que bares podem não ter proprietários, ou não sabermos quem é o proprietário, por isso o valor zero. A cardinalidade máxima igual a 1 determina que, caso exista alguma pessoa proprietária daquele bar, só pode ser uma única ocorrência na entidade Pessoa. Já a cardinalidade (0,N), próxima da entidade Bar, deve ser entendida da seguinte forma: nem toda pessoa é proprietária de algum bar, mas, caso seja, ela pode ter mais de uma propriedade.

A prática de modelagem conceitual considerando o MER utiliza apenas as cardinalidades máximas para especificar os tipos básicos de relacionamentos entre entidades. Temos, de maneira geral, três situações distintas:

- Relacionamento NxN
- Relacionamento 1xN
- Relacionamento 1x1

Note que, assim como para as entidades, **a modelagem ER permite atributos também para relacionamentos**. No exemplo do esquema conceitual ER das cervejas, o atributo Valor do relacionamento Vende permite especificar o preço que cada bar vende suas cervejas. Esse atributo não é nem da entidade Bar, pois não faz parte dos conceitos que descrevem um bar, nem é atributo da entidade Cerveja, pois cada marca pode ter um valor de venda diferente em cada bar em que é vendida.



Mantendo o foco!

Há situações na modelagem conceitual em que é necessário representar um **relacionamento que ocorre entre ocorrências de uma mesma entidade**. Esse tipo de relacionamento é **chamado de “autorrelacionamento”** e é muito comum na prática. No caso do esquema ER das cervejas, vemos na Figura 2.02B que estamos representando relacionamentos familiares na modelagem. Assim, há ocorrências de Pessoas que são filhas de outras ocorrências presentes nesse banco de dados. Analogamente, há mães e pais representados conceitualmente. Estamos supondo, por simplificação, que não temos pessoas nas diversas hierarquias familiares com mesmo nome; caso contrário, teríamos de adicionar algum atributo como CPF ou RG.

Um **conceito** complementar, presente em particular nos autorrelacionamentos, é a noção de **papel**, que ajuda na interpretação de relacionamentos. No caso do autorrelacionamento “Filho de” entre pessoas, temos os papéis de “pai-ou-mãe” e “filho” (ou filha) explicitados. Em termos de cardinalidades, a modelagem desse esquema ER em particular permite ocorrências da entidade Pessoa que não tenham vínculos familiares, isto é, não sejam pais ou filhos – cardinalidades mínimas iguais a zero. Já se alguém é mãe ou pai de alguém, pode ser de mais de uma ocorrência. Para simplificar a modelagem, estamos considerando que apenas uma mãe, ou um pai, são identificados no esquema para cada pessoa. Essa noção de papel pode ser explicitada em relacionamentos de qualquer tipo. É o caso, por exemplo, do papel de Proprietária junto da entidade Pessoa para o relacionamento “Dono de” referente a algum bar.

Para maior clareza conceitual das entidades e dos relacionamentos representados em um DER, é possível utilizar **um recurso** bem interessante **de modelagem, chamado de generalização ou especialização**. No DER, utilizamos um símbolo triangular para representar esse tipo especial de relacionamento, também conhecido por *IS-A* (“é um”, em inglês). Na Figura 2.02B temos um exemplo para a entidade Cerveja, especializada em duas outras entidades: Nacional e Importada.

Resumo

Dizemos que Cerveja é uma generalização das entidades Nacional e Importada, ou, ainda, que uma ocorrência na entidade Nacional é uma Cerveja. O mesmo vale para a entidade Importada. Na verdade, tudo depende de como foi o **processo de modelagem**: por um lado, se tínhamos apenas a entidade Cerveja e verificamos que, nesse exemplo, temos dados de fabricantes apenas de cervejas nacionais, para as demais que são importadas apenas os países de origem, decidimos criar as entidades especializadas. Por outro lado, podemos imaginar que o processo criou inicialmente duas entidades separadas, Cervejas Nacionais e Cervejas Importadas, com vários atributos em comum, apenas dois diferentes. Assim, generalizamos em uma entidade Cerveja tudo em comum, mantendo nas entidades especializadas o que é diferente. No final, independentemente do processo, o resultado é o mesmo. Cabe observar que se trata de uma maneira de representar hierarquias de conceitos também. Os atributos da entidade Nacional são {Fabricante, Marca, Cor e Tipo}, e os atributos da entidade Importada são {País_de_Origem, Marca, Cor e Tipo}. O identificador (atributo Marca) é o mesmo para a entidade genérica Cerveja, assim como para as entidades especializadas. As entidades especializadas

herdam os atributos da entidade genérica. Na Figura 2.02B temos, junto ao triângulo do relacionamento *IS-A*, entre parênteses, duas letras que detalham o tipo de hierarquia representado. O primeiro valor pode usar a letra t ou p, informando que se trata de hierarquia total ou parcial. No caso, usamos a letra t, pois toda cerveja ou é nacional, ou é importada. O segundo valor pode ser igual às letras d ou c, significando disjunto ou compartilhado. Disjunto significa que uma ocorrência da entidade genérica pode assumir no máximo uma ocorrência na entidade especializada, e compartilhado indica que poderia estar em mais de uma. Neste exemplo, assumimos apenas um papel, isto é, as cervejas são ora nacionais, ora importadas.



A modelagem conceitual usando o MER prevê a possibilidade de representação de relacionamentos múltiplos envolvendo três ou mais entidades. Os relacionamentos binários – entre apenas duas entidades – são os mais comuns em um DER, mas também podem existir relacionamentos ternários, com três entidades, não representados nesse exemplo das cervejas.

Por fim, outro recurso menos comum, mas também interessante, é a possibilidade de representar “relacionamentos entre relacionamentos” por meio do conceito de entidade associativa (ou “superentidade”). É uma forma de contornar uma restrição sintática no DER, que não permite ligar diretamente um relacionamento a outro relacionamento. Suponhamos que quiséssemos representar no esquema conceitual das cervejas uma nova entidade chamada Fatura, que registraria os dados referentes às vendas de cervejas por determinado bar sempre que algum cliente do bar solicitasse. A entidade Fatura não se relacionaria com a entidade Bar apenas, pois depende da venda da cerveja. Analogamente, não é o caso de um relacionamento com a entidade Cerveja, pois a fatura depende da ocorrência na entidade Bar. Como não ocorre sempre, não seria um relacionamento ternário. Logo, teríamos que considerar o relacionamento Vende entre Bar e Cerveja como uma “superentidade” contemplando atributos em Bar, Cerveja e Vende, se relacionando com a entidade Fatura.

Mapeamento MER para modelo relacional

A modelagem conceitual é o primeiro passo apenas no sentido de obtermos um esquema de banco de dados relacional. Sabemos que modelos conceituais não são modelos de dados por completo pois só definem a parte estrutural de representação dos dados, mas não têm manuseio de dados, como ocorre nos modelos (lógicos) de dados.

O mapeamento de um esquema conceitual, definido pelo MER, em um esquema lógico do modelo relacional envolve transformar cada elemento de um esquema de entidades e relacionamentos (entidades, relacionamentos e atributos) em elementos equivalentes da modelagem relacional (tabelas, atributos e restrições estruturais).

Uma transformação mais detalhada e completa, envolvendo todos os componentes presentes em um MER, pode ser consultada nas referências bibliográficas. A tabela a seguir exhibe uma síntese das principais transformações entre esquemas desses dois modelos.

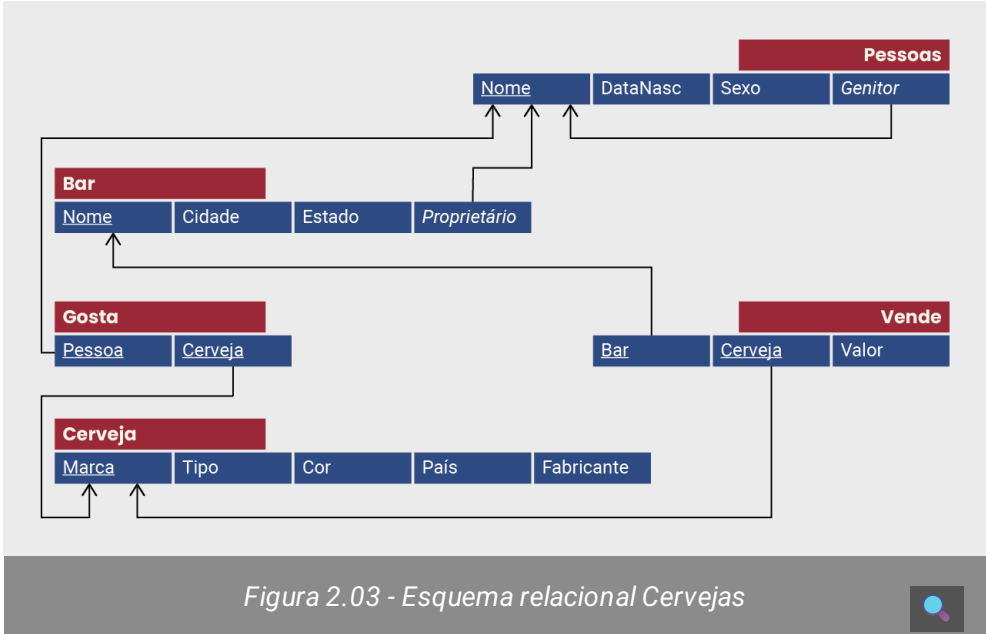


Modelo de entidades e relacionamentos	Modelo relacional
Entidade	Tabela
Atributo de entidade	Atributo de tabela
Atributo multivalorado	Tabela
Atributo estruturado	Tabela
Atributo determinante	Chave primária
Domínio	Domínio
Entidade fraca	Tabela com chave primária composta
Dependência de existência	Não há conceito equivalente no modelo relacional
Relacionamento	Chave estrangeira ou tabela
Relacionamento que identifica uma entidade fraca	Chave estrangeira, que é parte da chave primária da tabela criada para representar a entidade fraca
Relacionamento múltiplo	Tabela
Papel	Não há conceito equivalente no modelo relacional
Atributo de relacionamento	Atributo de tabela
Autorrelacionamento	Chave estrangeira ou tabela
Agregado	Chave estrangeira ou tabela
Generalização	Uma ou mais tabelas

Ao aplicarmos as transformações da tabela apresentada ao esquema conceitual das cervejas na Figura 2.02B, vamos obter um esquema relacional como o ilustrado na Figura 2.03. Como se trata do modelo relacional, temos agora apenas tabelas isoladas. Não há mais *links*, como temos com relacionamentos no DER.

Dado que há alternativas em algumas associações da tabela, mais de uma solução considerada correta poderia ser gerada nesse processo de mapeamento entre modelos. O importante é garantir que as informações existentes antes e depois da transformação sigam as mesmas, e é isso que essas regras de equivalências entre os componentes de cada modelo tentam realizar.

Note que na Figura 2.03, cada tabela tem sua própria chave primária (PK), e algumas podem ter também chaves estrangeiras (FK). As PKs estão sublinhadas: no caso da tabela Cerveja, é a coluna Marca, e, para a tabela Gosta, é uma chave (composta!) das colunas Pessoa e Cerveja. Já as integridades referenciais das FKs são apontadas pelas linhas com setas indicando quais atributos referenciam ou são referenciados. Por exemplo, a coluna Cerveja é FK na tabela Vende referenciando a coluna Marca na tabela Cerveja, assim como o atributo Proprietário na tabela Bar referencia o atributo Nome na tabela Pessoa.





Mas, finalmente, como chegamos ao esquema relacional a partir do esquema conceitual? Bom, inicialmente transformamos cada entidade em uma tabela com os respectivos atributos. Assim, foram criadas as tabelas Cerveja, Bar e Pessoa, inicialmente com os atributos das entidades apenas.

Ainda temos mais duas entidades, Nacional e Importada, presentes no esquema conceitual como parte do relacionamento *IS-A* com a entidade Cerveja, e que também devem ser mapeadas para o esquema relacional. Como podemos ver na imagem anterior, optamos aqui por mapear toda a hierarquia para uma única tabela, e, assim, mais dois atributos passam a fazer parte da tabela Cerveja: País (nome simplificado para o atributo “País de Origem” no conceitual) e Fabricante.

Nesses casos, os atributos identificadores das entidades foram mapeados diretamente para as chaves primárias das respectivas tabelas no esquema relacional. Cabe observar que nem sempre esse mapeamento é direto, pois no MER os identificadores podem ser equivalentes a superchaves do modelo relacional, não sendo minimais como exigido para chaves candidatas. Se, nesse caso do esquema conceitual das Cervejas, tivéssemos como identificador da entidade Pessoa o par de atributos Nome e DataNasc, a PK da tabela não seria composta, pois, por suposição, o atributo Nome sozinho já é capaz de identificar tuplas.

Aprenda mais

Todas as entidades foram devidamente mapeadas. Vamos agora analisar os relacionamentos presentes na Figura 2.02B. Como visto na tabela, há pelo menos duas formas de representar relacionamentos do MER no modelo relacional, dependendo de a cardinalidade ser NxN, 1xN ou 1x1.

Com a criação de uma tabela para representar o relacionamento.

Neste caso, a chave primária da tabela que representa o relacionamento é definida pela combinação dos identificadores das duas entidades que participam desse relacionamento. No modelo relacional, cada parte dessa chave primária será uma chave estrangeira. Essa alternativa é típica de relacionamentos **NxN**.

Com a criação de uma chave estrangeira em uma das tabelas.

Essas chaves estrangeiras na tabela representam as entidades envolvidas no relacionamento. Essa situação é comum para relacionamentos **1xN**.

Os relacionamentos de cardinalidade 1x1 podem ser considerados casos particulares dos anteriores e são mais raros. A opção preferida, normalmente, é a fusão das entidades envolvidas no relacionamento em uma única tabela, incluindo todos os atributos, inclusive os do relacionamento. Porém, também há casos em que o procedimento para relacionamentos 1xN é mais adequado. Não temos relacionamentos 1x1 em nosso exemplo da Figura 2.02B; logo, vamos nos concentrar nos outros dois casos que são mais comuns nas modelagens de aplicações reais.

Inicialmente, vamos analisar o relacionamento “Dono de” entre Pessoa e Bar. Nesse caso, por ser do tipo 1xN (cardinalidades



máximas!), basta considerar a PK da tabela Pessoa como atributo adicional na tabela Bar, transformando um *link* explícito no DER em um relacionamento orientado por valores. Optamos por chamar de Proprietário esse novo atributo, que naturalmente passa a ser FK em Bar, referenciando a PK Nome em Pessoa.

Em seguida, temos o relacionamento Vende, entre Bar e Cerveja. A cardinalidade, nesse caso, é NxN. Assim, passamos a ter a tabela Vende no esquema relacional, contendo os atributos PK oriundos das tabelas Bar e Cerveja. Como o relacionamento tem atributo, este também se transforma em coluna da tabela respectiva, mas não fará parte da PK. Para facilitar a interpretação, o atributo Nome da entidade Bar passa a se chamar Bar na tabela Vende, assim como o atributo Marca de Cerveja, que renomeamos para Cerveja. A PK da tabela Vende é o par (Bar, Cerveja), e ambos os atributos da PK são, separadamente, FKs para as tabelas Cerveja e Pessoa, respectivamente.

Mantendo o foco!

No caso do relacionamento Gosta, temos um procedimento similar ao relacionamento Vende, porém mais simples, pois não temos atributos presentes no relacionamento. Logo, são apenas duas colunas que, juntas, formam a PK da tabela Gosta e, separadamente, são FK para as tabelas Cerveja e Pessoa.

Por fim, temos o autorrelacionamento “Filho de” para a entidade Pessoa que precisa ser também ser mapeado para o esquema relacional. Como não deixa de ser um relacionamento 1xN, podemos seguir a mesma regra anterior de mapeamento, o que irá gerar uma situação interessante: a chave estrangeira apontará para uma chave primária na própria tabela, e não uma tabela diferente, como acontece normalmente. É necessário modificar o nome do atributo FK, senão ele teria o mesmo nome da PK. No esquema relacional Cervejas, da Figura 2.03, temos a adição do atributo Genitor (nome alternativo para mãe ou pai) na tabela Pessoa, que será uma FK referenciando a PK Nome na mesma tabela.

Criando tabelas com a linguagem SQ



Uma vez completo o mapeamento do esquema conceitual para o esquema lógico, é necessário implementar as tabelas em um SGBD utilizando a fração DDL da linguagem SQL, que permite criar os objetos do banco de dados relacional.

Um sistema de banco de dados relacional pode conter diversos objetos, como tabelas, visões e índices. Nesta aula, a ênfase é dada na construção das tabelas, que são a base de qualquer sistema relacional. Para isso, temos na imagem a seguir a sintaxe básica de criação de tabelas por meio do comando *CREATE TABLE*. Para cada tabela é preciso informar o seu nome, suas colunas com o domínio de valores válidos e, também, as restrições de integridade estruturais.

CREATE TABLE <nome-da-tabela>
(<lista e domínio das colunas>,
 <restrições de integridade estruturais>)

Restrições estruturais (modelo relacional)

- Chave primária
 - CONSTRAINT <nome restrição>
 PRIMARY KEY (<atributo(s) PK>)
- Apenas controle de valores únicos
 - CONSTRAINT <nome restrição>
 UNIQUE (<atributo único>)
- Integridade semântica simples
 - CONSTRAINT <nome restrição>
 CHECK [expressão lógica com <nome-atributo>]
- Chave estrangeira
 - CONSTRAINT <nome restrição>
 FOREIGN KEY (<atributo(s) FK>) REFERENCES <atributo(s)-PK-tabela-referenciada>

DOMÍNIOS

- Char (N) / Varchar (N)
- Integer, Numeric(6,2)
- Date
- ...

Figura 2.04 - SQL DDL – criação de tabelas



Cabe, neste momento, relembrar a importância das regras ou restrições de integridade no modelo de dados relacional. Para bem definir um esquema relacional particular, e garantir que teremos no banco de dados apenas as informações úteis e desejadas, é necessário especificar restrições que controlam a integridade dos dados que se deseja representar.

Como vimos na aula anterior, existem, por um lado, os controles inerentes à estrutura do modelo de dados considerado. Por outro lado, há preocupação com regras que são mais genéricas e associadas à aplicação em si. Sendo modelo relacional o modelo lógico escolhido, as restrições são definidas em função de os dados serem representados na forma de relações ou tabelas. Já no segundo caso, contemplam-se as chamadas regras de negócio (*business rules*), que são independentes do modelo de dados lógico adotado. Podemos citar como exemplo a regra que diz que um aluno só pode fazer no máximo 30 créditos por semestre em um banco de dados acadêmico de uma universidade, ou, ainda, a restrição para um esquema relacional de uma empresa genérica que diz que nenhum empregado pode ganhar menos que o salário mínimo, nem ganhar mais do que seu chefe imediato.

Valores válidos nos atributos

Na aula anterior, estudamos a restrição de chave, que diz respeito ao controle de unicidade de tuplas, contemplando as noções fundamentais de chaves candidatas e primárias. Depois, foi mencionada a restrição de integridade referencial, que diz respeito às chaves estrangeiras. Entretanto, no caso do modelo relacional, temos outras restrições também consideradas estruturais: **as restrições de entidade e de domínio**.

Fique ligado

A qualidade de um banco de dados relacional passa pelo controle de valores válidos em suas tuplas para cada atributo presente no esquema. Além de evitar elementos duplicados, também é possível especificar os valores esperados em função da semântica do atributo determinado pela aplicação em si.

A restrição de integridade de entidade diz respeito ao fato de que valores nulos não podem ser considerados nas chaves primárias. Como não há como diferenciar dois nulos, estes não podem ser valores válidos nos atributos da PK de uma tabela. Ao definirmos que um atributo faz parte da chave primária de uma tabela, automaticamente estamos impondo uma regra de valores nulos não serem aceitos, garantindo a unicidade das tuplas, como esperado.

Cabe observar que há SGBDs que aceitam nulos em uma PK mesmo com a existência dessa restrição em todos os livros de referência da área. A justificativa é que os valores *NULL* assumem, internamente, valores numéricos distintos, e, assim, haveria como controlar a unicidade. Entretanto, uma das grandes vantagens dos sistemas de bancos de dados relacionais é fazer com que os usuários possam abstrair dos detalhes de implementação e armazenamento dos dados. Logo, não há, na prática, como diferenciar dois nulos, e recomenda-se fortemente considerar a restrição de entidade como regra em qualquer situação.

As colunas em cada tabela estão normalmente associadas com domínios de valores válidos e esperados. Há casos relativamente



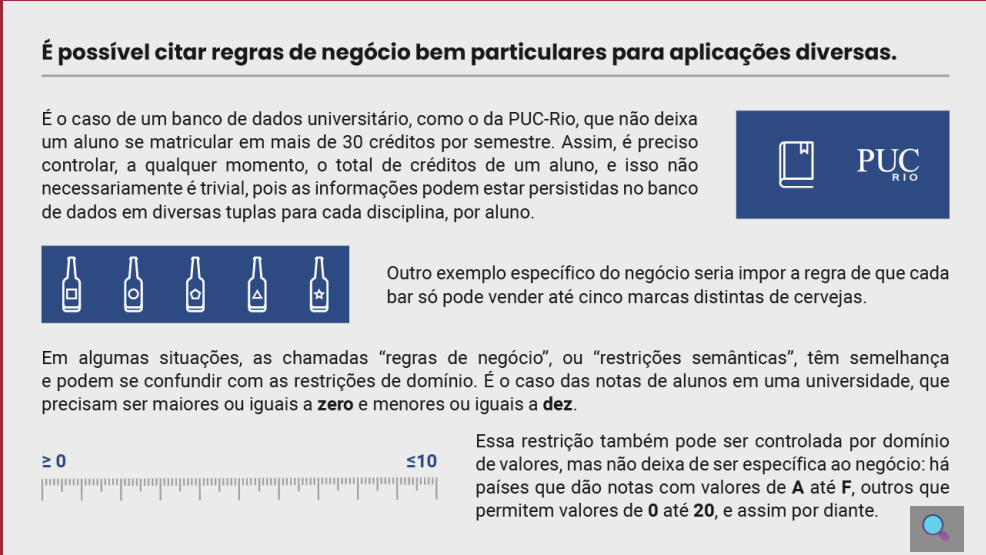
simples, em que valores para o atributo Nome, em uma tabela Pessoa, não podem exceder certa quantidade de caracteres, limitando-se, em alguns casos, aos caracteres alfabéticos. Em outras situações, podemos ser bem mais específicos: a coluna Data de Nascimento deve ser necessariamente anterior à data corrente do sistema, ou, ainda, o valor atribuído às notas de uma prova deve ser numérico, porém restrito à escala de 0 a 10. Ao se definir cada tabela em um banco de dados relacional, e suas respectivas colunas, **a restrição de integridade de domínio** é aquela que busca definir as regras de valores válidos para cada atributo.

Regras de negócio

Também não se pode deixar de mencionar aquelas restrições que não fazem nenhuma referência à estrutura do modelo de dados em questão. No caso do modelo relacional, estamos nos referindo às regras que impõem qualidade aos dados do banco de dados, mas que não dependem do fato de termos tabelas para representá-los.

Aprenda mais

O infográfico a seguir mostra exemplos práticos de regras de negócio, que precisam ser consideradas em seus respectivos bancos de dados.



Diferentemente das regras de integridade estruturais, não basta definir tabelas e atributos para impor as restrições desejadas no caso do controle semântico das aplicações particulares. Os SGBDs relacionais oferecem assertivas e outros objetos, como funções, procedimentos armazenados (*stored procedures*, ou SP) ou gatilhos (*triggers*) que permitem utilizar instruções SQL combinadas com comandos tipicamente presentes em linguagens de programação convencionais para especificar as regras de negócio. Esse assunto é considerado um tópico mais avançado no estudo de sistemas de bancos de dados e pode ser estudado consultando-se as referências bibliográficas aqui citadas.

Fração DDL da linguagem SQL

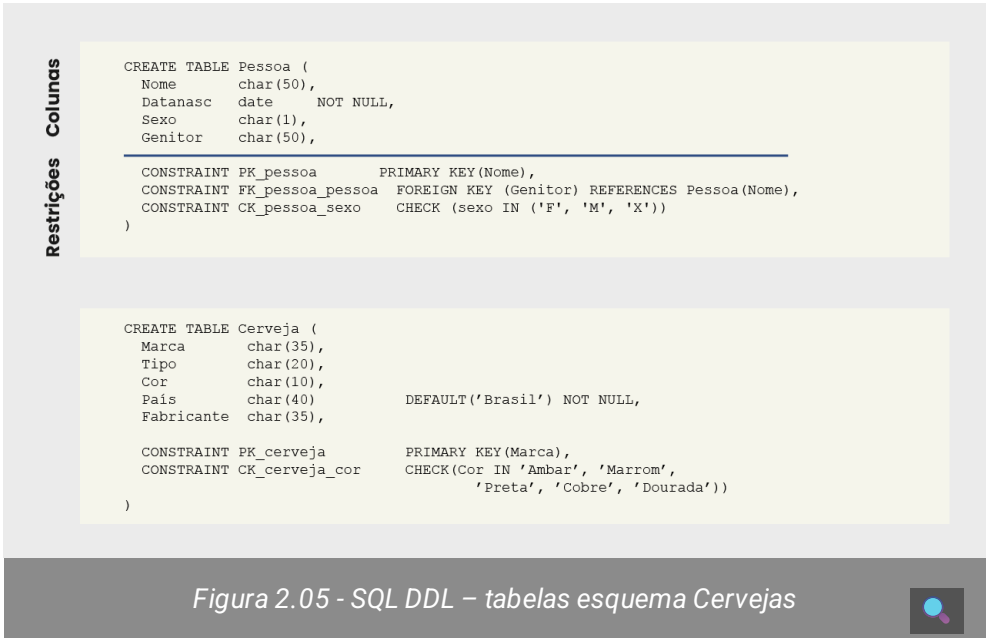
Voltando ao comando *CREATE TABLE* da Figura 2.04, temos que a linguagem oferece alguns tipos de dados que permitem controlar domínios de valores para cada coluna da tabela. Por exemplo, para atributos numéricos existem tipos como *INTEGER* e *NUMERIC*, que não somente garantem que apenas dígitos sejam utilizados, que



expressões aritméticas possam ser usadas, mas que também podem controlar o total de algarismos, a parte inteira e decimal de um número.

Se precisamos armazenar uma cadeia de caracteres (*strings*) qualquer, envolvendo letras e números, podem-se utilizar os tipos CHAR e VARCHAR. Basicamente, a diferença diz respeito ao fato de que o tipo CHAR(10) reserva sempre 10 caracteres para representar a *string*, mesmo que tenhamos palavras com até 5 caracteres apenas, enquanto VARCHAR(10) apenas impõe um limite máximo e só ocupa, em *bytes*, o estritamente necessário. Apesar de vantajoso em termos de espaço, o manuseio de estruturas de tamanho fixo sempre é mais eficiente de maneira geral. Assim, há vantagens e desvantagens para esses dois tipos adaptados para *strings*.

Na Figura 2.04 temos também exemplos das sintaxes básicas para definição de restrições estruturais, como as chaves primárias e as chaves estrangeiras. Para facilitar a compreensão, ilustramos, nas imagens a seguir, as expressões do comando *CREATE TABLE* utilizadas para criar as tabelas do esquema relacional Cervejas, obtido a partir do processo de mapeamento do modelo conceitual para o modelo lógico.



A Figura 2.05 traz os comandos de criação das tabelas Pessoa e Cerveja. Em termos sintáticos, assim como na fração DML de consultas da SQL, só é necessário ter espaços em branco entre comandos e objetos do banco de dados. Para fins didáticos, nesses exemplos temos, então, separadas as colunas de cada tabela e as demais restrições.

Fique ligado

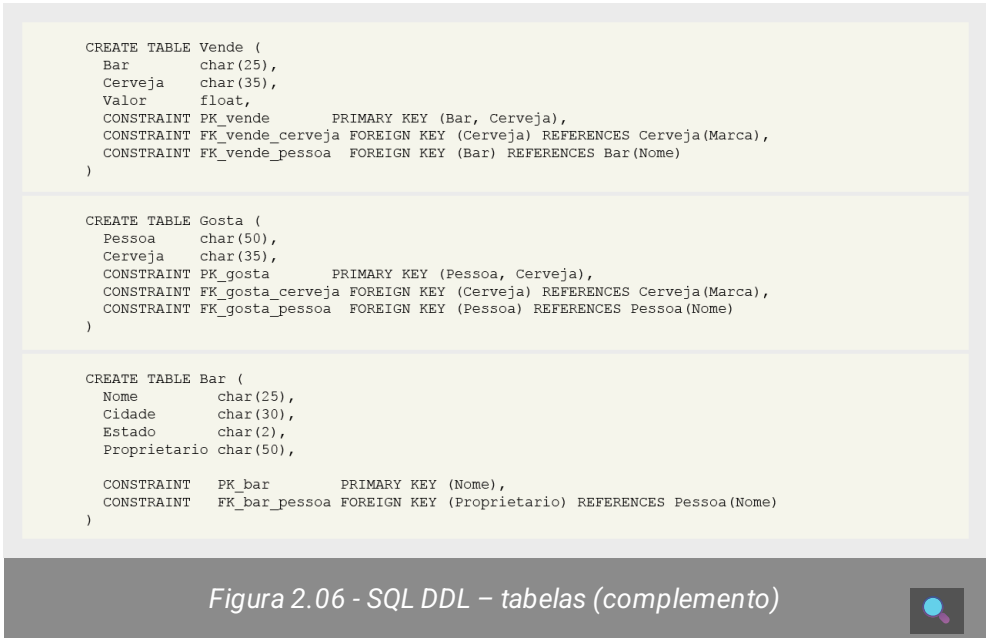
Para a tabela Pessoa, temos quatro colunas, e as duas colunas que fazem referência aos nomes de pessoas ocupam 50 caracteres – uma escolha arbitrária nesse caso; poderia ser mais ou menos –, mas ambas precisam ter a mesma quantidade de caracteres pela restrição de FK para PK. A coluna DataNasc (data de nascimento) é associada ao tipo DATE, que garante que valores nessa coluna serão datas válidas, além de permitir o uso de funções sobre datas que são úteis em aplicações reais. Por exemplo, é possível obter a idade de qualquer pessoa ao se comparar a data corrente do sistema com a data de nascimento. Poderemos também fazer consultas filtrando todas as pessoas que nasceram em determinado mês. Ou seja, atribuir o tipo de dado apropriado garante integridade e vantagens no manuseio daqueles valores.

A última coluna da tabela Pessoa é a Sexo, que, para esse exemplo em particular, escolhemos representar por apenas um caractere com o tipo Char(1). Entretanto, as restrições (*constraints*) definidas permitem ir além: nesse caso só seriam aceitos no banco de dados valores iguais a F, M ou X, respectivamente feminino, masculino e neutro, ou não declarado. Vale observar que, segundo a Figura 2.02B, todos os atributos, exceto a PK, poderiam aceitar valores nulos, pois DataNasc e Sexo estão definidos como (0,1), logo, são opcionais. Entretanto, por motivo exclusivamente didático, optou-se na criação da tabela pela exigência de valores válidos para DataNasc, ilustrando o uso da sintaxe *NOT NULL* no comando *CREATE TABLE*. Por *default*, o padrão é aceitar nulos, e, nesses casos, a linguagem SQL DDL não exige explicitar *NULL* ao lado de cada atributo. De todo modo, na prática esta situação é muito comum: a modelagem lógica (relacional) acrescentar (ou relaxar) restrições presentes na modelagem conceitual (ER).

Por último, temos as definições das restrições estruturais do modelo relacional mais conhecidas: o atributo Nome é considerado a PK da tabela, e o atributo Genitor é FK referenciando a própria tabela Pessoa nesse caso, já que é oriundo de um autorrelacionamento. É recomendado, mas não obrigatório, nomear as restrições de integridade, como fazemos na Figura 2.05. Por exemplo, o nome da restrição *CK_pessoa_sexo* será exibido na tela do computador sempre que algum usuário tentar colocar algum dado nessa coluna, para determinada tupla, que não satisfaça os valores F, M e X. Em um banco de dados com muitas restrições criadas, não há dúvida de que facilita termos mensagens de erros que identificam onde foi o problema.

Ainda na Figura 2.05 temos o comando *CREATE TABLE* da tabela Cerveja. Além de outras colunas e restrições particulares, com características similares às da tabela Pessoa, temos um exemplo de valor *default* para a coluna País, associando o valor Brasil sempre que uma inclusão de tuplas na tabela não explicitar qual é o país de origem da cerveja.

Na figura a seguir, temos os comandos de criação das três tabelas restantes no esquema relacional da Figura 2.03. Cabe observar apenas o fato de que as tabelas Vende e Gosta têm, ambas, PK compostas por dois atributos. Além disso, cada atributo participante da PK também é FK para alguma PK em outras tabelas do esquema.



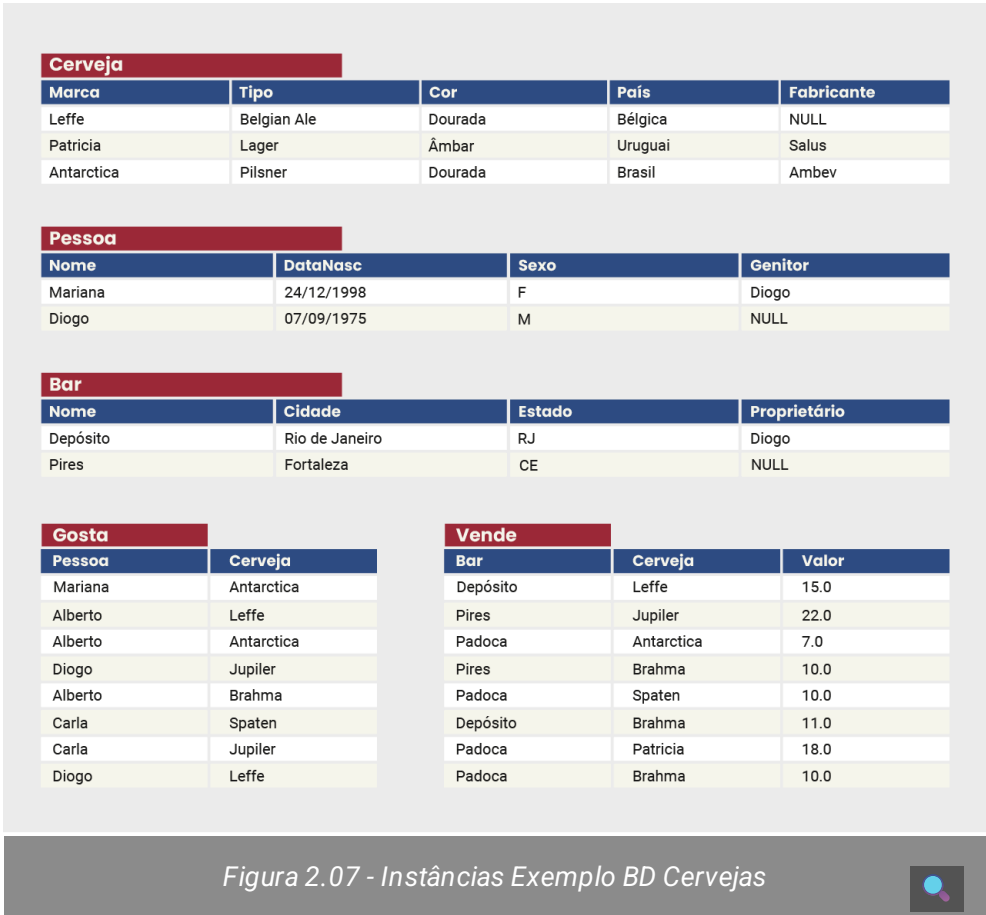
Cabe observar que essa parte DDL (*data definition language*) da linguagem SQL é dedicada não às instâncias (os dados), e sim às estruturas que definem o esquema do banco de dados. Se estamos manuseando o esquema, vale comentar sobre dois outros comandos SQL muito usados em aplicações reais. Existe a expressão *ALTER TABLE*, que permite alterar nomes e tipos de colunas, adicionar ou retirar restrições de integridade e fazer outras modificações na

estrutura das tabelas. Interessante observar que muitos comandos *ALTER TABLE* podem ser feitos mesmo com dados já presentes nas tabelas. Podemos citar a situação particular da criação da FK Genitor, na tabela Pessoa, que referencia o atributo Nome na mesma tabela. Há SGBDs que não aceitam o comando *CREATE TABLE* como consta na Figura 2.05: é exigido primeiro criar a tabela sem chave estrangeira e, em um momento posterior, utilizar *ALTER TABLE* para definir a integridade adequada.

Por fim, assim como criamos, podemos querer excluir tabelas do nosso esquema e, para isso, devemos utilizar o comando *DROP TABLE*. Sugerimos a leitura dos livros e manuais dos SGBDs *on-line* para mais detalhes de cada um desses comandos.

Manuseio de dados nas tabelas

Com as tabelas criadas e implementadas, o próximo passo é incluir os dados propriamente ditos no banco. A imagem a seguir ilustra essa situação, com todas as tabelas do esquema relacional das cervejas contendo algumas tuplas que permitem compreender melhor o banco de dados que será disponibilizado para usuários e sistemas. Mas como foi que esses dados foram inseridos ou disponibilizados?



Veremos, a seguir, os comandos SQL que viabilizam a carga dos dados em um banco relacional. Nesse ponto, voltamos a utilizar a linguagem em sua fração DML (*data manipulation language*), cujo foco são as instâncias, e não mais o esquema, como é o caso da fração DDL. Porém, diferentemente da primeira aula, não vamos praticar consultas, apenas **as instruções que nos permitem inserir, atualizar e excluir dados nos esquemas relacionais**.

O comando básico para inserir dados em um esquema relacional é o INSERT. Vamos adotar aqui a mesma metodologia de aprendizagem por exemplos, sem separar sintaxe da semântica da linguagem. A figura a seguir ilustra diversos comandos de inserção de dados com a linguagem SQL. Basicamente, é necessário escolher qual tabela deve receber dados e, pela cláusula VALUES, explicitar os valores.



Um dos exemplos, com destaque em amarelo, chama a atenção para o fato de que, dependendo da implementação do SGBD em particular, valores de datas precisam respeitar o padrão da instalação feita com o SGBD. Não retorna erro, mas é importante saber disso para evitar interpretações erradas de dias e meses. No caso da imagem anterior, a intenção era inserir “24 de dezembro”. O SGBD utilizado nesses exemplos (PostgreSQL) considera datas na ordem mês, dia e ano, assim como vários outros SGBDs de mercado.

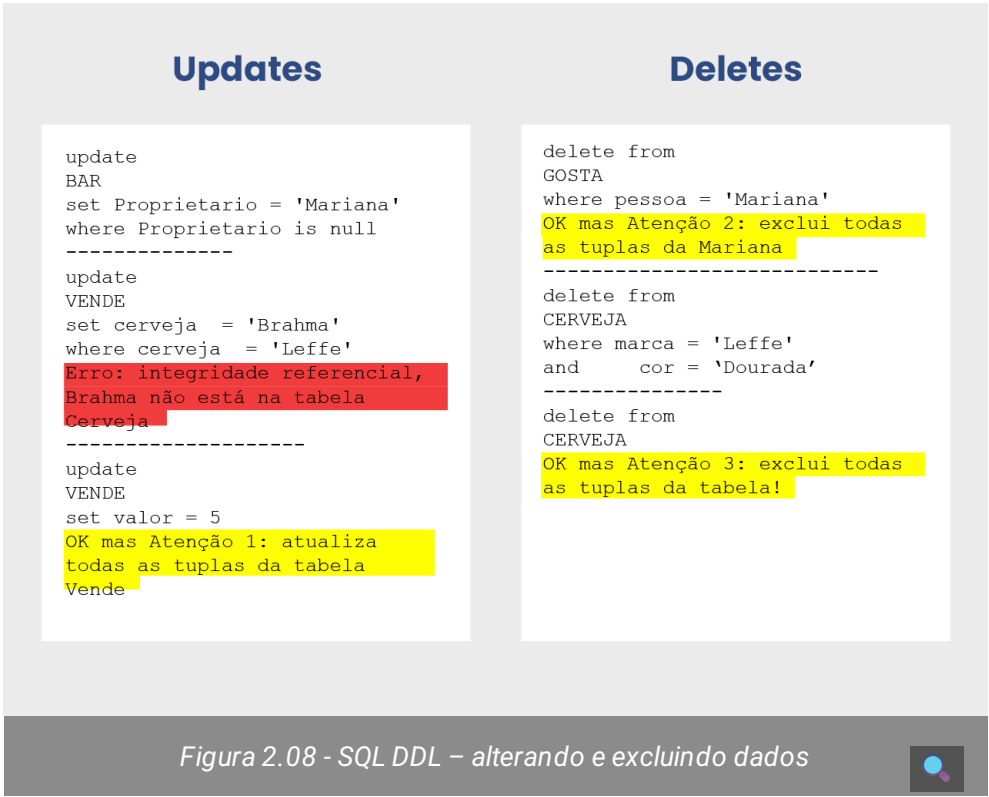
Os demais exemplos, com destaque em vermelho, têm explicitadas as situações de erro. O erro #1, por exemplo, exhibe o controle de valores únicos por meio da PK, que não pode aceitar nulos (integridade de entidade). Já o erro #2 aconteceria pelo fato de a criação da tabela ter pré-definido os valores possíveis e *NULL* não estar entre esses valores. O controle de integridade por meio de FK para cervejas inexistentes como PK em alguma tabela seria ativado no caso do erro #3. Por fim, exibimos um exemplo da vantagem de definirmos domínios apropriados aos atributos, por controlarem não só o tipo de caractere, mas também alguma semântica do dado em si. Nesse caso do erro#4, o fato de definir uma coluna como tipo DATE controla também quais datas são válidas, sem que o usuário precise desenvolver programas específicos para isso.

Fique ligado

Usamos o comando INSERT para a carga inicial de dados e posterior manutenção do banco de dados com novas informações. Entretanto, como em qualquer sistema, pode ser necessário alterar algum dado ou mesmo excluí-lo, por conta de erros de manuseio ou adequação ao sistema em uso. Como em qualquer linguagem de programação que usa arquivos, podemos sempre proceder com alterações nos dados por meio de uma exclusão seguida de nova inserção. Entretanto, para grandes volumes em sistemas de bancos de dados, isso pode ser inviável. Assim, a linguagem SQL oferece, além do comando DELETE, que exclui tuplas, o comando UPDATE, que permite alterar dados persistidos, mantendo o controle de integridade em cada operação.



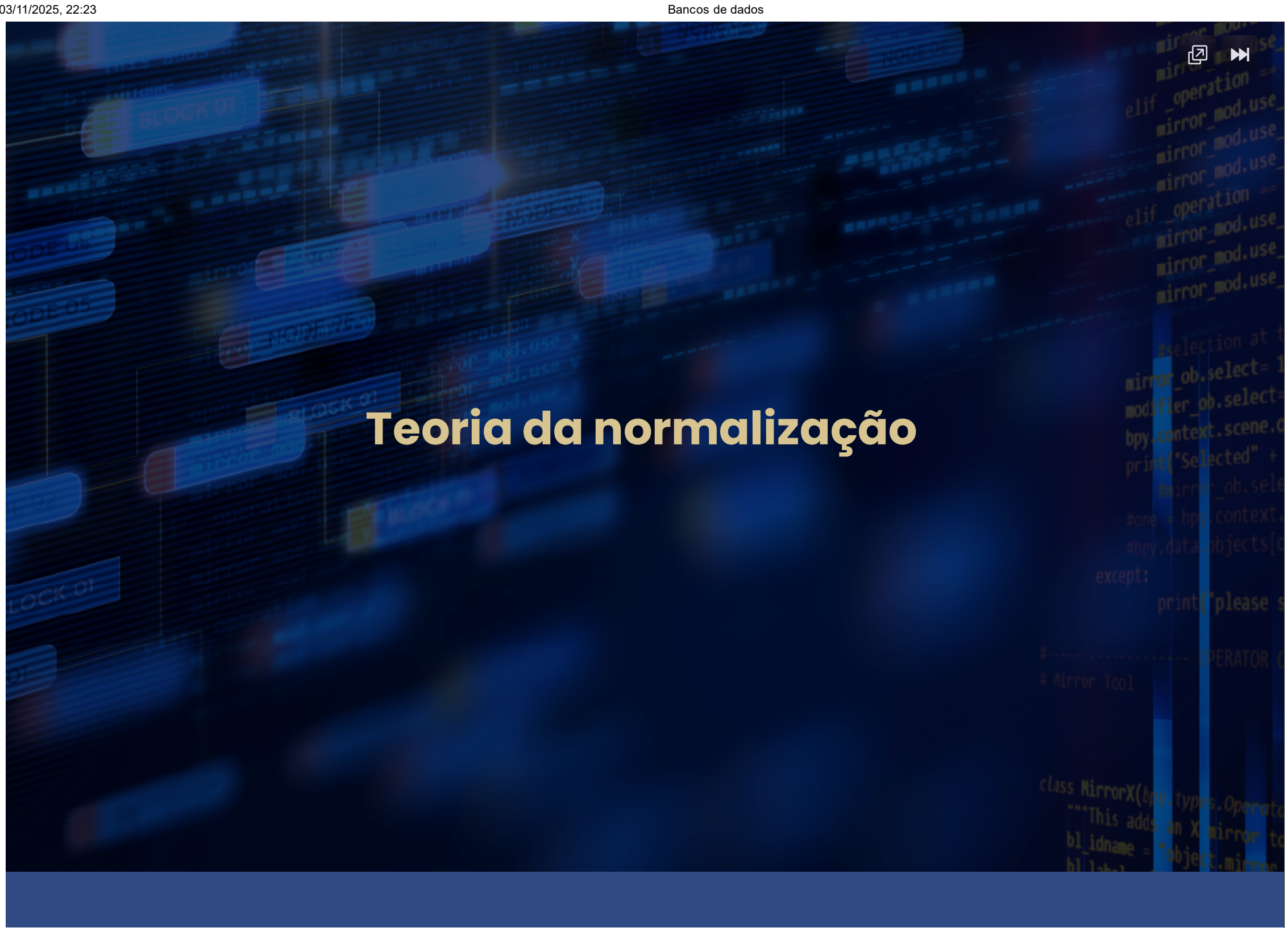
Para o esquema relacional das cervejas, temos, na imagem a seguir, alguns exemplos que ilustram a situação de atualizações de dados particulares. Mais adiante, quando colocar “a mão na massa”, o aluno poderá explorar outros casos distintos e conhecer em mais detalhes as opções sintáticas para os comandos UPDATE e DELETE e também o comando INSERT, discutido anteriormente.



Nessa imagem, temos algumas expressões que seriam aplicadas ao banco de dados sem maiores problemas e outras que merecem atenção ou mesmo retornariam com erro pelo controle de integridade do SGBD. Por exemplo, ao tentarmos mudar o nome de uma cerveja em todas as tuplas da tabela Vende, teríamos uma mensagem de erro de integridade referencial caso o novo valor para cerveja (FK na tabela Vende) não conste nos valores da PK da tabela Cerveja.

Os demais casos não levantariam erro de controle do SGBD, mas, fica a dica: os comandos UPDATE e DELETE, sem cláusula WHERE, aplicam a atualização correspondente a todas as tuplas da tabela. Em particular, deve-se ter muito cuidado com o uso do comando DELETE, sobretudo em um sistema de produção. Mesmo quando a cláusula *WHERE* está presente é preciso lembrar do *modus operandi* dos sistemas relacionais, que são orientados aos conjuntos (*set-oriented*): mesmo que a intenção de determinado comando seja eliminar apenas uma tupla em particular, o SGBD vai aplicar o predicado lógico em todas as tuplas que validam a expressão correspondente ao *WHERE*. Na imagem anterior, esse seria o caso do DELETE para pessoas de nome “Mariana”.

Cabe comentar sobre uma situação prática que um SGBD relacional também monitora para fins de controle de integridade: se algum comando DELETE ou UPDATE excluir ou alterar o valor de alguma coluna PK de uma tabela que é referenciada por alguma FK da mesma ou de outra tabela, pode acontecer que o SGBD faça atualizações complementares automaticamente, em sequência ou “em cascata” (*delete cascade* ou *update cascade*) após a exclusão ou alteração. Por exemplo, uma ou mais tuplas da tabela Gosta faz referência à cerveja da marca Leffe de cor Dourada na tabela Cerveja; a operação de DELETE correspondente na Figura 2.08 implicaria o SGBD automaticamente alterar para *NULL* todos os valores na FK da tabela Gosta, mantendo o banco de dados íntegro. O comportamento efetivo do comando SQL é definido por cada SGBD de maneira independente. Este é somente um exemplo.



Teoria da normalização

Neste momento temos, então, um percurso de projeto de banco de dados relacional praticamente completo, saindo de um esquema puramente conceitual (MER) até um esquema relacional inteiramente definido por suas tabelas e restrições estruturais. Como consta na Figura 2.01, temos um caminho alternativo para chegar ao mesmo resultado por meio da **teoria da normalização**. Cabe observar que, historicamente, esta foi a primeira abordagem de modelagem lógica proposta e discutida pelos pesquisadores e desenvolvedores no início dos anos de 1970. A abordagem via modelagem conceitual, que acabamos de ilustrar, ganhou força apenas na segunda metade da década de 1970 após a ampla disseminação do modelo de entidades e relacionamentos.

De imediato, vamos explicar sucintamente os fundamentos e conceitos principais da teoria da normalização, a começar pela importante noção de **dependência funcional**.

Sejam:

- R (A1, A2, ... An) uma relação (tabela) com N colunas;
- X e Y subconjuntos de {A1, A2, ... An}, os atributos de R;
- $t_1, t_2, \dots, t_m$ as tuplas presentes em R.

Dizemos que $X \rightarrow Y$ é uma notação para representar que X determina Y

(ou, analogamente, Y depende funcionalmente de X)

Se, e somente se, $t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y], \forall t_1, t_2$ em r
instância de R

Para o nosso esquema relacional das cervejas, temos que, por exemplo:



- (tabela Pessoas) Nome → DataNasc

- (tabela Cerveja) Marca → País, Tipo, Cor

Assim, as dependências funcionais são restrições sobre os atributos e tuplas em todas as tabelas de um banco de dados relacional que devem ser validadas para qualquer instância.

Para qualquer esquema relacional, como é o caso do esquema das cervejas, teremos um conjunto F de dependências funcionais (DF) contemplando os diversos atributos ou colunas das tabelas. Um DBA ou analista de sistemas, ou mesmo os programadores da aplicação, devem ser capazes de definir esse conjunto de DFs que vão permitir uma especificação mais precisa das regras de integridade, que devem ser verificadas permanentemente.

Vamos definir agora o conjunto F+ como o Fecho (*closure*) do Conjunto F das DFs válidas para uma ou mais tabelas de um esquema relacional. Na verdade, F+ corresponde ao conjunto de todas as DFs implicadas logicamente pelo conjunto F.

Por exemplo, se temos para uma tabela específica que:

- F: {A → B; B → C}

Podemos dizer que:

- F+: {A → B; B → C; A → C}

Por transitividade, se A determina B e B determina C, é possível inferir uma nova DF válida para a tabela contemplando os atributos A, B e C. Ou seja, não é necessário definir todas as DFs que possam ser válidas para um esquema relacional.



Para demonstrar uma utilidade da noção de DFs para o modelo relacional, estamos em condições de formalizar a noção de **chave**: sejam $R(A_1, A_2, \dots, A_n)$ uma tabela qualquer com n atributos; F um conjunto de DFs válidas para R ; e $X \subseteq \{A_1, A_2, \dots, A_n\}$. Podemos dizer que X é chave de R se:

- $X \rightarrow \{A_1, A_2, \dots, A_n\} \in F^+$; e
- não existir $Y \subset X$ tal que $Y \rightarrow \{A_1, A_2, \dots, A_n\} \in F^+$.

Isto é, para ser chave, é preciso que exista uma DF em F^+ com X determinando (funcionalmente) todos os demais atributos da tabela e não termos outro subconjunto Y de atributos de R , que seja, por sua vez, subconjunto de X , para o qual tenhamos também uma DF em F^+ determinando todos as colunas de R . A primeira condição diz respeito à propriedade de uma ou mais colunas serem consideradas superchaves de R . Já a segunda condição é a de minimalidade, exigida para chaves candidatas e primárias.

Como vimos na primeira aula, sabemos que, se há mais de uma chave para R , elas são chamadas de “chaves candidatas”. Uma delas é escolhida arbitrariamente, ou seguindo conhecimentos da aplicação, como chave primária (PK) da tabela.

Essa formalização do conceito de chave permite definir também a noção de atributo primo (*prime*), que é qualquer atributo da tabela R que é parte de alguma chave candidata. Vimos, na primeira aula, um exemplo clássico de chaves candidatas com uma relação chamada Pessoa (que poderia ser uma extensão da relação Pessoa do esquema das cervejas), contemplando atributos como nome, CPF, RG (Registro Geral) e UF (unidade federativa, ou estado). Nesse caso, temos uma chave candidata $K_1 = \{CPF\}$ e outra chave $K_2 = \{RG, UF\}$. Os atributos CPF, RG e UF são, então, chamados de “atributos primos”, pois são todos atributos de alguma das chaves candidatas.

Cabe observar que existem algoritmos que usam essa definição formal baseada em dependências funcionais para obter as PKs das tabelas em um esquema relacional, mas esses procedimentos estão fora do escopo desta aula.

Ao mesmo tempo, as DFs também podem ser usadas no processo de projeto de um esquema relacional, em que começamos com uma relação (universal) contendo todos os atributos envolvidos no banco de dados e vamos decompondo em tabelas menores até obter um conjunto de tabelas ditas normalizadas, que atendem especificações de qualidade conhecidas como “formas normais”.



Projeto de banco de dados lógico

Neste momento, convidamos os alunos a acompanharem mais explicações sobre esses conceitos neste vídeo



Não é simples determinar qual é o conjunto “ótimo” de tabelas para um esquema relacional em particular. Existem muitas situações distintas do uso do sistema de banco de dados que nem sempre conseguimos contemplar integralmente. Assim, mesmo um esquema relacional normalizado, considerado de “boa qualidade”, pois atende determinado conjunto de formas normais, pode ser eficaz e correto, porém não eficiente, no sentido de desempenho. Quando discutirmos, na próxima aula, os aspectos relacionados com projeto físico, veremos que uma das técnicas mais conhecidas para obter esquemas relacionais com boa *performance* envolve aplicar, para algumas tabelas, um processo chamado “desnormalização”.

Transações em bancos de dados



Agora que já conhecemos a linguagem SQL tanto para consultas como para criação de tabelas e atualização de dados, podemos concluir esta aula comentando sobre o importante conceito de transações em bancos de dados relacionais.

Uma transação é considerada uma unidade básica de trabalho. Transações são tão importantes para os sistemas de bancos de dados como processos, ou tarefas, o são para os sistemas operacionais mais populares (p. ex., Linux, Windows).

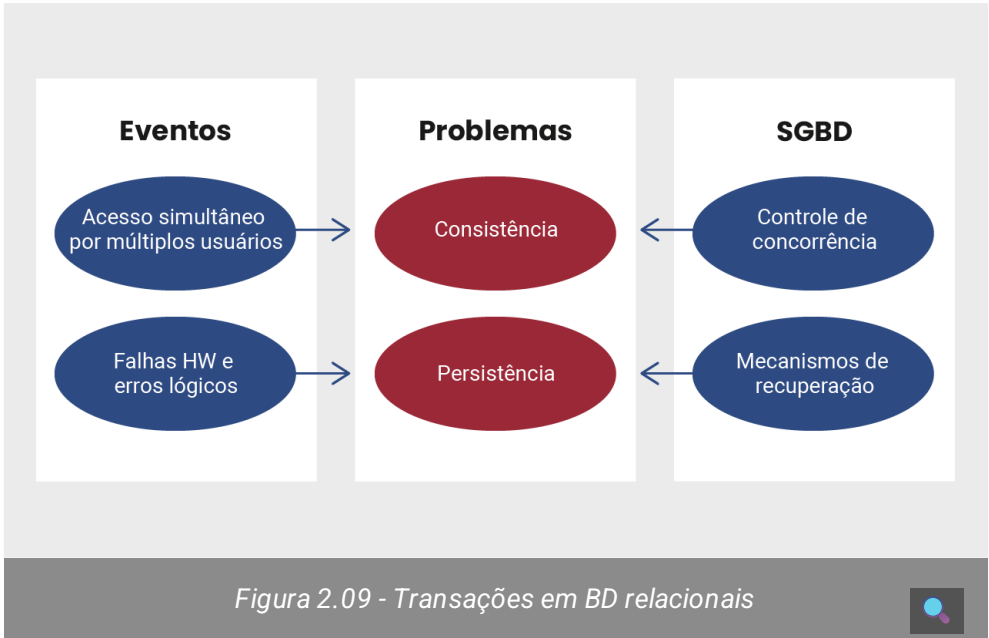


Uma transação é, então, uma unidade de execução de um programa que acessa e atualiza vários itens em um banco de dados. Trata-se de um conjunto de instruções em SQL que é criado para atender determinada funcionalidade no sistema em operação. O exemplo canônico diz respeito a transações com operações bancárias: se queremos fazer uma transferência de valores entre uma conta poupança e uma conta-corrente, precisamos definir uma transação que, primeiro, retira valores da conta poupança para, em seguida, inserir valores na conta- corrente. São duas operações básicas (um DELETE, seguido de um INSERT) nas estruturas persistentes correspondentes às tabelas do banco de dados em questão.

Com o intuito de preservar a integridade, um sistema de banco de dados, com apoio de um SGBD, deve garantir as seguintes propriedades, conhecidas pelo acrônimo ACID (atomicidade, consistência, isolamento e durabilidade):



Se pudéssemos optar por oferecer um sistema de banco de dados em que cada transação (de um mesmo ou múltiplos usuários) é executada uma de cada vez, como se estivessem em uma fila de prioridade, não teríamos necessidade de gerenciar e controlar transações para manter a integridade do sistema. Entretanto, sistemas de bancos de dados foram pensados para uso de múltiplos usuários e transações simultâneas, visando ao uso efetivo dos recursos computacionais disponíveis.



Na figura apresentada, ilustramos os eventos típicos que podem causar problemas em um sistema de banco de dados relacional. Os problemas de consistência e persistência têm a ver com as propriedades ACID, e, assim, os SGBDs relacionais contemplam algoritmos e técnicas para controle de concorrência e mecanismos de recuperação após falhas.

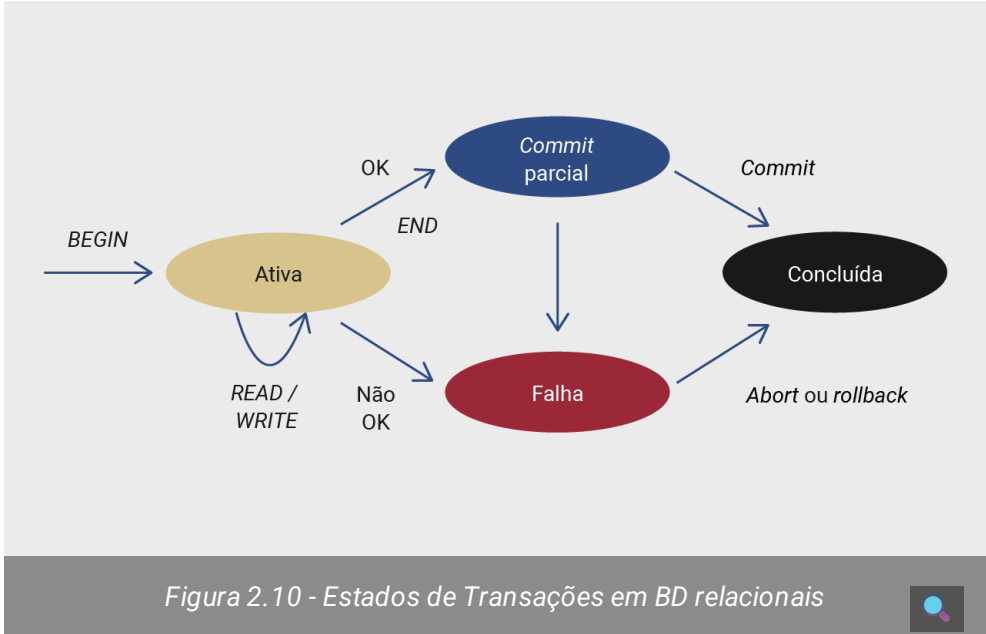


Figura 2.10 - Estados de Transações em BD relacionais

Para concluir, ilustramos, na Figura 2.10, o “ciclo de vida” de uma transação em um banco de dados relacional. Todo bloco de operações básicas contido em uma transação é iniciado pelo comando *BEGIN* e termina pelo comando *END*. De maneira geral, as informações importantes dizem respeito às operações que fazem acesso em leitura (*READ*) ou gravação de dados (*WRITE*) presentes em cada expressão contida na transação.

Toda transação sempre termina, com ou sem sucesso. Mesmo quando tudo dá certo e consideramos “OK”, a transação fica em um estado chamado de *COMMIT* parcial. Em inglês, *COMMIT* é sinônimo de compromisso, ou seja, estamos aguardando que outras transações concorrentes terminem para podermos concluir com um *COMMIT* final com sucesso. Caso outra transação que tenha gravado valores lidos pela transação em *COMMIT* parcial tenha terminado com falha, o estado da transação muda de *COMMIT* parcial para FALHA. Nesses casos, a transação é abortada com operadores específicos de cancelamento ou *ROLLBACK*, uma operação da linguagem SQL que permite retornar ao estado consistente inicial do sistema de banco de dados.

Religião

A construção de um esquema relacional completo, com tabelas, atributos e restrições de integridade, é uma maneira de garantir um sistema de banco de dados de acordo com as especificações e objetivos iniciais. Entretanto, com o esquema criado, normalmente faz-se uma carga de dados massiva, para que o banco de dados tenha utilidade prática. Sabemos que com as PK definidas em cada tabela, as inserções de novos dados podem não ser rápidas, já que o SGBD precisa controlar a integridade implementada. Assim, muitos DBAs optam por “desligar” o controle de integridade no momento de popular o banco de dados. Entretanto, sem as PKs, podemos ter muitos elementos repetidos, o que contraria a teoria do modelo relacional. Pode ser até mesmo inviável reativar (“religar”) o controle de tuplas sem repetição. Se você tiver o poder de decisão na sua empresa ou projeto individual, qual seria sua estratégia? Quais são as vantagens e as desvantagens de cada alternativa?

Mais: considere uma situação em que temos que fazer 1 milhão de comandos *INSERT* e, para isso, iniciamos uma transação para carga no banco de dados. Suponha que, após 999 mil inserções, há um *INSERT* que gera uma mensagem de erro, com o SGBD bloqueando a operação por conta de um valor duplicado na chave. No modelo transacional dos sistemas relacionais, temos que a propriedade de atomicidade exige “tudo ou nada”, e, nesse caso, teríamos perdido tempo e incomodado todos os demais usuários do sistema à toa, pois tudo teria de ser desfeito. Como você resolveria uma situação dessas?



Referências

Clique aqui para acessar as referências e créditos desta aula.

